

Apply to Infrastructure — How It Works

What this feature does

After a simulation passes validation (`allowed: true`), the UI shows an “**Apply to Infrastructure**” button for `add_compute` and `remove_node` actions. Clicking it makes the change *real* — not on a real physical rack, but on the actual DigitalOcean droplets that stand in for the two racks in this project. This is the one place in the system where a user action leaves the sandbox and touches live cloud infrastructure.

The physical model

There are only two real DigitalOcean droplets, each acting as one “rack”:

Rack ID	Droplet public IP	Docker network	IP range
<code>droplet-1-tor1</code>	<code>165.22.221.170</code>	<code>docker-compose_tor1-net</code>	<code>10.10.1.0/24</code>
<code>droplet-2-tor2</code>	<code>157.245.107.150</code>	<code>docker-compose_tor2-net</code>	<code>10.10.2.0/24</code>

Servers and routers are not separate droplets — they are Docker containers running *inside* one of these two droplets, attached to that rack’s Docker network. So “add a compute node to rack 1” really means: SSH into the `droplet-1-tor1` droplet and start a new container on `tor1-net` .

Request flow

1. **User runs a simulation** and it comes back `allowed: true` for `add_compute` or `remove_node` .
2. **User clicks “Apply to Infrastructure”** in the confirmation dialog (`ApplyDialog` in `PromptAssistant.tsx`), which is a distinct, explicit second confirmation — separate from running the simulation itself.
3. **Frontend calls** `POST /api/v1/apply/simulate` with `{ action, params }` (the same action/params the simulation was run with).

4. **Backend (`api/routes/apply.py`) dispatches** to `_apply_add_compute` or `_apply_remove_node` .

Add compute (`_apply_add_compute`)

1. Resolve which rack (`droplet-1-tor1` or `droplet-2-tor2`) the new node belongs to, from `target_droplet` / `target_rack_id` , or by mapping the router ID back to its rack.
2. **Fetch the droplet's current public IP live from the DigitalOcean API** (not from Terraform state) — droplets get rebuilt/recreated and their IPs go stale in `terraform.tfstate` , so the DO API is the source of truth. If the DO API call fails, it falls back to whatever IP is recorded in `terraform.tfstate` .
3. **SSH into that droplet** (`root@<host>` , using a deploy key checked into `terraform-pipeline/hpe-twin-deploy-key` , falling back to `~/.ssh/id_ed25519` for local dev).
4. Ask the droplet, via `docker inspect` , which IPs on its rack's subnet are already taken, and pick the next free one (starting at `.11`).
5. Check no container with that name already exists (`docker ps -a --filter name=...`) — if it does, return `409 Conflict` rather than silently overwriting it.
6. Run `docker run -d --name <name> --network <rack-net> --ip <next-free-ip> --cap-add NET_ADMIN --restart unless-stopped ubuntu:24.04 sleep infinity` — a lightweight placeholder container representing the new compute node.
7. Return the container name, host, rack, and assigned IP back to the UI.

Remove node (`_apply_remove_node`)

1. Since the request doesn't say which rack the node lives on, the backend **checks both droplets** (`docker ps -a --filter name=...`) to find which one is actually running a container with that name.
2. Before deleting, it captures a snapshot of the container's network config (`docker inspect ... NetworkSettings.Networks`) — this is what makes revert possible later.
3. Runs `docker stop <name> && docker rm <name>` .
4. Returns the captured snapshot alongside the result so the frontend can offer “Revert.”

Revert path

Every apply response includes exactly what's needed to undo it. The `ApplyDialog` keeps this in state and offers a “**Revert Change**” button:

- **Reverting an `add_compute`** just stops and removes the container that was created (`_revert_add_compute`) — clean and total, since nothing else depended on it.
- **Reverting a `remove_node`** recreates a container with the same name on the same rack/network, but with a **freshly assigned IP** (not necessarily the original one) — this is a best-effort restore, not a perfect undo, since the original container's exact state (mounted volumes, running processes, etc.) is not preserved. It's the placeholder-image approach again: a fresh `ubuntu:24.04 sleep infinity` container with the same identity in the topology.

`POST /api/v1/apply/revert` takes the `action` , `container_name` , `host` , and `rack` returned by the original apply call, so the frontend doesn't need to re-derive anything.

Why only `add_compute` and `remove_node`

Every other action type (`move_server` , `inject_compute` , `inject_network` , `inject_storage` , `migrate_rack`) is either a purely structural simulation concept (there's no real “move” to perform against placeholder containers) or a synthetic stress injection meant to test the *validator*, not something that should ever be pushed to real infrastructure. The apply router explicitly rejects any other action with `400 Bad Request` .

Failure handling

- **SSH failures** (`_ssh` helper) distinguish connection timeout (`504`), non-zero exit code (`502` with the container's stderr), and missing SSH client (`500`) — so a failed apply tells you *why* it failed, not just that it did.
- **Unknown rack** → `400` .
- **Can't resolve a droplet's public IP** (both DO API and Terraform state come up empty) → `502` , listing which racks *were* resolvable.
- **Container name collision on add, or missing container on remove** → `409` / `404` respectively, rather than a generic failure.

Security notes

- The DigitalOcean API token is read from `terraform-pipeline/terraform.tfvars` first, falling back to the `DO_TOKEN` environment variable — never hardcoded.
- SSH uses `StrictHostKeyChecking=no` because droplet IPs can change across rebuilds and there's no static known-hosts entry to check against — acceptable here because this is an internal sandbox environment with a deploy key scoped to it, not a general-purpose production pattern.
- `BatchMode=yes` ensures a failed/missing key fails fast instead of hanging on an interactive password prompt.